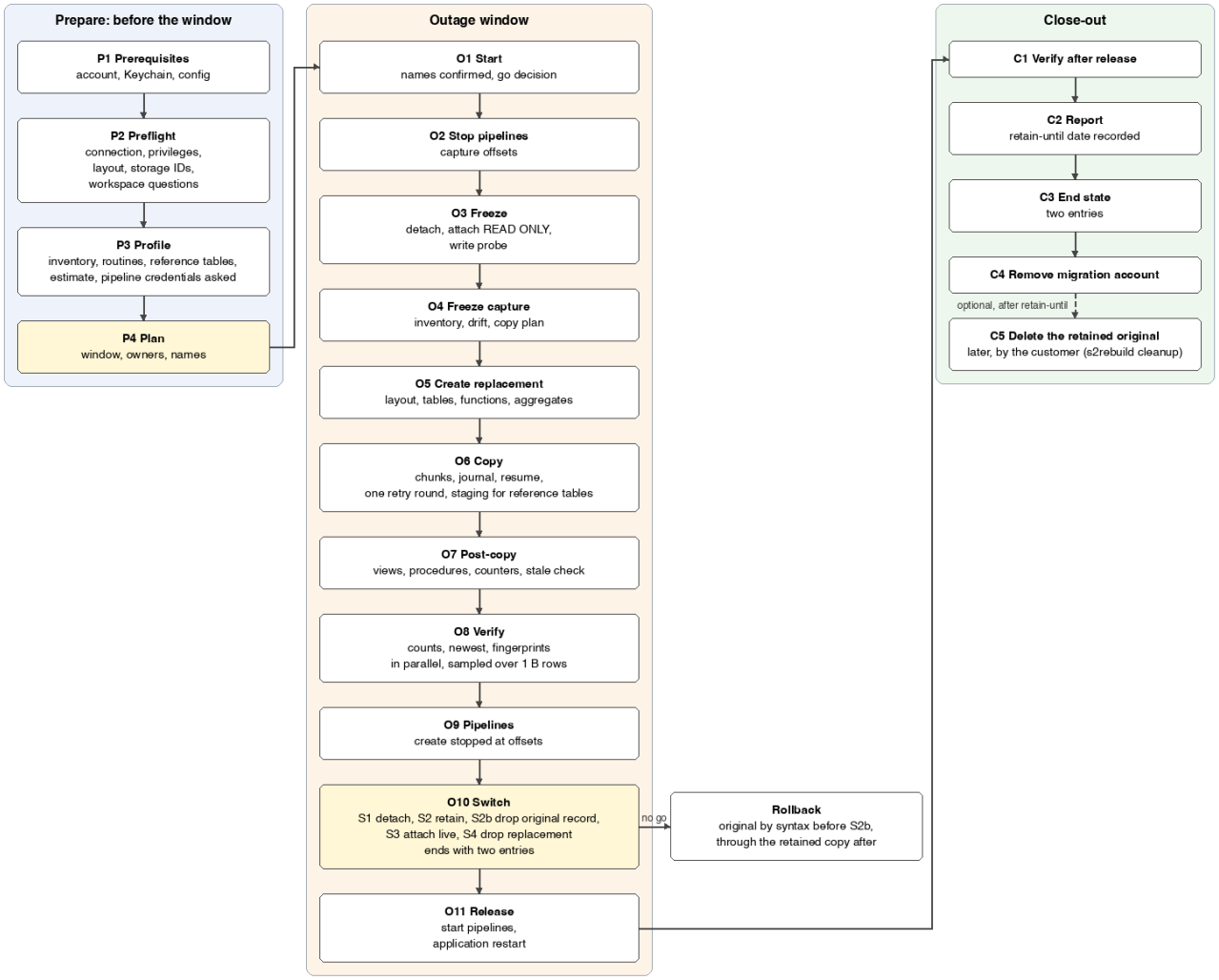
SingleStore Database Rebuild Kit

Step guide: the README and the step catalog

Author: John Bergamini

Version 1.0.0

Flow of the steps



SingleStore Database Rebuild Kit

Author: John Bergamini. Version 1.0.0. Proprietary; see LICENSE.

The kit rebuilds a SingleStore Helios database into a new partition layout and cuts it over under its original name inside one outage window. The first outage step makes the source read only. Everything after that is a copy, a verification and a switch, each behind a gate, each restartable from a terminal. A run ends with two entries on the workspace: the live database in its new layout, and the original kept read only under a retained name until the customer deletes it.

Who runs what

Identity	Who	Steps
runner	A SQL account the kit creates at P1 and drops at C4	P2, P3 tables, O4, O5 database and tables, O6, O7 counters and stale check, O8, C1, C2
owner	The workspace admin SQL user, or the Portal identity in the owner notebook	P1 account, P2 and P3 pipelines, routines and write-capable accounts, O2, O3, O5 functions and aggregates, O7 views and procedures, O9, O10, O11, C3, C4, cleanup
operator	The person running the kit	every command; types the owner password, the pipeline credentials and the names when the kit asks; records the gates a person decides
customer	The application owner	stops the application (if asked), supplies the pipeline credentials, answers the workspace questions (autoscale, auto-suspend), restarts the application at O11, decides when the retained original is deleted (C5)

The runner copies. The owner freezes, switches and releases. With an owner profile in the configuration the kit runs the admin steps itself; without one it prints each admin block and waits for `s2rebuild gate`.

Prerequisites

Checked by `s2rebuild check` (P1 and P2) unless the table says otherwise.

Need	Who provides	Checked by
A Helios workspace with the source database attached READWRITE, engine 8.9 or later (tested on 10.8.0)	customer	P2
Workspace state ACTIVE, autoscale off, auto-suspend off or a keep-alive job for the window	customer	O1 records the operator's answer (`--workspace NOTE`)
Compute size: leaf cores times 8 at or above the target partition count; an S-8 at 4X copied 17.7 B rows in about 5 hours with 12 workers	customer	P2
An admin SQL user for the workspace (the owner); its password in the Keychain item named in the configuration, or typed at P1	customer	P1
The runner account (`connections.runner.user`, `s2rebuild_runner` in the example), created by the kit at P1 with a password that meets the workspace policy; privileges SELECT, INSERT, DELETE, CREATE, DROP, ALTER, CREATE DATABASE, DROP DATABASE, PROCESS, CREATE VIEW, SHOW VIEW on `*.*`	kit	P1 creates, P2 verifies each privilege by name

Owner profile in the configuration, so admin steps execute with gates; without it the kit prints blocks and waits for recorded gates, and routines, pipelines and other accounts' grants are not visible	customer	P2
Pipeline credentials per source type (S3 key pair, Kafka SASL user and password, or whatever the pipeline CONFIG names)	customer	P3 asks; a blank answer skips the type with a recorded decision
Free names for the replacement and the retained database	customer or the kit's date default	P2; O1 asks in production mode when the configuration leaves them out
A named owner for "application stopped" and "application restarted"	customer	P4, recorded at O1
Operator machine: Python 3.10 or later, `pip install -e .` (PyMySQL, PyYAML, openpyxl), the macOS Keychain or environment variables, a network path to the workspace host on port 3306 with TLS, the SingleStore CA bundle in `certs/`	operator	P1
A clone of the kit at a release tag; `customer/config.yaml` written from `examples/config.example.yaml`	operator	P1
Disk: a few hundred MB per run for the journal, inventory and reports	operator	none
The operator's Mac on power and awake for the whole night: the kit starts `caffeinate -i -w <pid>` for every `run` and `step`; between commands run `caffeinate -i` in a terminal	operator	none

Not required: an outage window (optional), a notebook, the Portal during the run, SUPER (cannot be granted on Helios).

Install

Python 3.10 or later on macOS or Linux. The Keychain password source is macOS only; elsewhere use `password_env`.

```
python3 -m venv .venv
.venv/bin/pip install -e .
.venv/bin/s2rebuild --help
```

Run the offline tests with `.venv/bin/pip install pytest` and `.venv/bin/python -m pytest`. No test opens a network connection.

Configure

1. Copy the example: `cp examples/config.example.yaml customer/config.yaml`.
2. Set the workspace host, the database names and the target layout. Every key is in docs/CONFIG.md.
3. Passwords: with `password_keychain` the kit fills the macOS Keychain items itself at P1 (`s2rebuild account-sql --execute` asks for the owner password when its item is missing, hidden, and generates the runner password to the workspace policy); or store them yourself (`security add-generic-password -a "$USER" -s s2rebuild-owner -w`), or name an environment variable with `password_env`. Pipeline credentials are asked for at P3 (or O9) and stored in the items named in `pipelines.credentials_keychain`; the variables in `pipelines.credentials_env` stay the fallback. The kit never prints, logs or writes a password or credential.

4. Names: `databases[].replacement` and `retained` may be left out. In `run.mode: production` (the default) O1 asks for them with the date default as the suggestion; `run.mode: test` (or `--mode test`) applies the defaults silently.
5. Everything a run writes stays under `customer/`. The rest of the folder is the static tree.

Steps

Each step prints what it did and the gate it needs. Exit codes: 0 done, 1 refused, 2 incomplete (rerun to resume), 3 verification mismatch, 4 gate pending or failed. The full catalog with inputs, outputs, statements and recovery is docs/STEPS.md.

Every command works on one run under `customer/runs/`. The P1 gate starts the run and prints its id. Export that id as `S2REBUILD_RUN`, so every later command names the same run. Without `S2REBUILD_RUN` or `--run ID`, a command uses the most recent run. `s2rebuild run` resumes that run and skips the steps that passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names the new run). Once a run exists, `s2rebuild run --run ID` refuses an id that names no run, so a mistyped id never starts another run. Every gate hint and every log header prints the run id with its `--run` form.

The sequence from P1 to the go and on to O2, where RUN_ID is the id the P1 gate printed:

```
s2rebuild account-sql
s2rebuild gate P1 --pass --note TEXT
export S2REBUILD_RUN=RUN_ID
s2rebuild check
s2rebuild profile
s2rebuild plan
s2rebuild gate P4 --pass --note TEXT
s2rebuild run --from O1
s2rebuild gate O1 --pass --note TEXT
s2rebuild run
```

The first line prints the runner account block for the admin; `s2rebuild account-sql --execute` runs it instead with the owner profile (P1 below). After O11: `s2rebuild step C1`, `s2rebuild step C2`, `s2rebuild step C3`, `s2rebuild step C4`. Later, when the customer decides: `s2rebuild cleanup`.

Prepare (before the window)

1. P1 Prerequisites. Print the runner account block with `s2rebuild account-sql` and have the admin run it, or run it with `s2rebuild account-sql --execute`: it asks for the owner password when its Keychain item is missing, prints the workspace password policy, generates a runner password that meets it, stores it in the runner's Keychain item and creates the account. Write the configuration. Gate: the account, the Keychain items and the configuration are in place; record it with `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`.
2. P2 Preflight. `s2rebuild check` connects as the runner (and the owner), checks every runner privilege by name, records the source's branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), confirms the replacement and retained names are free and the compute size fits the partition count, and prints `copy.workers` (1 to 32) beside the leaf cores. It prints three questions (ACTIVE, autoscale off, auto-suspend off or a keep-alive job) for O1 to record. With the owner profile it counts the routines by kind; without it the line says they are not visible to the runner. It refuses until P1 is recorded. Run it again any time: O1 needs P2 passed in the same run. Gate: every line ok.
3. P3 Profile. `s2rebuild profile` captures the inventory (tables, keys, sizes, accounts, sessions, views; with the owner profile the pipelines, the routines and the write-capable accounts, which the runner cannot

see) and writes `profile.xlsx` with the estimate: one line, the rows divided by `estimate.rows_per_hour` (set the figure a run on your workspace measured). It refuses until P2 passed. Once P3 passed, a new capture needs `--redo`, which logs what it rewrites. `--dry-run` writes only `profile-dry-run.xlsx`, never the capture. Reference tables are counted exactly and planned through a staging table. Routines (functions, table-valued functions, aggregates, procedures) are captured with their definitions and recreated at O5 and O7; `objects.routines.ignore` leaves one behind on purpose. A pipeline that loads into an ignored procedure refuses the capture, since O9 would find no target. On route `kit` the kit then asks for the pipeline credentials of each source type (S3 key pair and token, Kafka SASL user and password, or key and value pairs for another type) and stores the CREDENTIALS JSON in the Keychain item named in `pipelines.credentials_keychain`; a blank answer skips the type, and the plan sheet and O9 list the pipelines that will not be recreated.

4. P4 Plan. `s2rebuild plan` prints the plan sheet: the window (when configured), step owners, the estimate line, pipeline credential route, the three workspace questions, the routines by kind, the retention days. Gate: the decisions are recorded with `s2rebuild gate P4 --pass --note TEXT`, the auto-suspend choice for the waits between commands among them.

Outage window

5. O1 Start. `s2rebuild run --from O1` checks that P2 passed in this run, records the window when one is configured (no step stops on it), records the configuration hash, asks for a replacement or retained name the configuration leaves to the kit (`run.mode: production`; `--mode test` applies the defaults), prints both names, and waits for the go decision: `s2rebuild gate O1 --pass --note TEXT`. `--workspace NOTE` records the answers to the three workspace questions as evidence of the go. Then `s2rebuild run` resumes the same run: it skips O1 and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.
6. O2 Stop pipelines. The owner stops every pipeline and the kit captures every definition and offset again after the stop. These are the offsets O9 replays. Gate: no pipeline Running; loaded and successful offsets agree on every partition; capture written.
7. O3 Freeze. DETACH, ATTACH READ ONLY, a write probe that must be refused. A DETACH that waits longer than `freeze.detach_wait_seconds` is cancelled and issued again with KILL. After a notebook or hand run, `s2rebuild step O3 --redo` runs the probe and records freeze_time. A run that stopped between DETACH and ATTACH resumes at the ATTACH: `s2rebuild step O3 --redo` sees the detached storage and the free source name and never detaches again. For a hand run, `s2rebuild owner-sql O3` writes that block to `owner-sql/O3-reentry.sql` beside `O3.sql`. The Portal may show 0 rows for the frozen source; the kit's counts are the reference. Gate: probe refused and READONLY.
8. O4 Freeze capture. `s2rebuild run` captures the inventory after the freeze and reports drift against P3 (tables, columns, views, routines and pipelines; the pipelines are not listed on the READ ONLY source and the report says so). `s2rebuild profile --step O4` does the same by hand and stays for a rerun: once O4 passed, a new capture needs `profile --step O4 --redo`, which rewrites the copy plan. Both refuse until O1 to O3 passed. Gate: drift reviewed.
9. O5 Create replacement. The run continues (or `s2rebuild run --from O5`): the replacement is created with the target layout and every table definition is compared with the capture; its STORAGE_ID is recorded. Then the owner session creates the captured functions and table-valued functions, and the aggregates after the functions they name; the gate compares each definition with the capture. A routine body that names the source database refuses the step before any statement.
10. O6 Copy. Server-side INSERT ... SELECT in hash buckets, journaled, resumable; a reference table goes through a staging table in the replacement. Stop it only with `s2rebuild pause`: a killed client leaves a

prepared transaction holding a leaf table lock for 15 to 20 minutes. The chunks that fail are retried once at the end of the pass, each no sooner than 20 minutes after its failure. The progress line names any chunk that runs longer than `copy.slow_chunk_minutes`. Gate: every chunk done; per-table count proof all match.

11. O7 Post-copy. Views, then procedures, AUTO_INCREMENT counters and the stale check (counts and newest values) with re-copy of anything that moved. A completed re-copy marks O6 passed again, and the counters are set after the last re-copy. A view runs under its definer and C4 removes the runner, so the runner never creates one. With the owner profile the kit creates the views, keeping each original definer whose account still exists, then the procedures the same way (a procedure may read a view, and a pipeline INTO PROCEDURE at O9 needs its procedure). Without it O7 writes `owner-sql/O7.sql` and ends pending: run that block as the admin, then `s2rebuild step O7`. A view or procedure whose definition names the source database refuses the step. Gate: views and procedures created, zero mismatches.
12. O8 Verify. Counts, newest values and grouped fingerprints, tier 3 in parallel on `verify.sessions` connections, largest table first. A table over `verify.full_fingerprint_max_rows` (1 billion rows) is sampled as a contiguous range on its sort key or time column, the same predicate on both sides; `verify.fingerprint_budget_minutes` is an elapsed deadline after which a table is recorded not fingerprinted. Gate: tiers 1 and 2 pass for every table; tier 3 a full match everywhere, or a sampled or not fingerprinted result accepted with `s2rebuild step O8 --yes`, which accepts the recorded result without a rerun when no chunk was copied since. `step O8 --full TABLE` or `--fingerprint TABLE --percent N` reruns tier 3 for one table. A mismatch marks the buckets for re-copy; run `s2rebuild run --from O6` to copy them again and rerun O7 and O8.
13. O9 Pipelines. The kit reads each pipeline's credentials from its Keychain item (filled at P3), else from the variable exported for this step only; a source type still without credentials is asked for here, and a blank answer skips it. The kit recreates every other pipeline in the replacement, stopped, at the O2 offsets. Route customer: the customer runs the placeholder file and the kit checks. Gate: count (the skipped pipelines left out), none Running, offsets equal the O2 capture partition by partition.
14. O10 Switch. A state machine that ends with two entries: S1 detaches the source; S2 attaches the original storage read only under the retained name; S2b attaches the original record by its own syntax under the live name, proves it unchanged (the three smallest tables, no foreign session) and drops it; S3 attaches the replacement as a branch under the live name; S4 drops the replacement record; the gate lists both storages and expects exactly the live record and the retained record. A failed proof at S2b stops with the original attached read write under the live name, the rollback state. S0 refuses unless every planned table was verified after its last chunk was copied; without the owner profile O10 refuses on the same rule, and the printed block and the notebook cell start with a STOP while it holds. A re-copy rewrites `owner-sql/O10.sql` with a STOP; generate the blocks or the notebook again after O8 passes. Type the live name when asked (or pass `--yes`). Gate: the live name resolves to the new branch with the target layout and the capture counts. From S3 the live name accepts writes. A rollback (`s2rebuild owner-sql O10 --rollback` writes both legs) runs only while every table still holds the row count of the copy and O11 has not started; it detaches the live name only while it holds the recorded live record, then re-attaches the original record by its own ATTACH_SYNTAX before S2b ran (R2a), or after S2b attaches a read-write branch of the retained copy under the live name (R2b). No rollback step drops the retained copy. The count check cannot see an update that keeps the count, so a rollback after S3 is safe only when the application stayed stopped until O11, as the O1 note records.
15. O11 Release. Retention check per partition, pipelines started; the customer restarts the application. Gate: application writing.

Close-out

16. C1 Verify after release. `s2rebuild step C1`: writer sample, count changes, pipelines Running with offsets advancing, application-managed pipelines back, no errors, grants compared. Gate: all ok and application writes seen. For a live database idle on purpose, pass `s2rebuild step C1 --idle-workload NOTE`. Without the owner profile the pipeline and grants checks are recorded as not checked.
17. C2 Report. `s2rebuild step C2` writes `final.xlsx` and `run-report.md` (the latest verification result per table, the two entries the run leaves with their records and ATTACH_SYNTAX, the listings before and after the switch and the end state, the retain-until date with the C5 command, the definer per view and routine, the workspace answers, the estimate against the actual rows per hour; a value the journal lacks reads "not recorded"). When the kit cannot pass C1 or C2, record that step by hand with `s2rebuild gate ID --pass --note TEXT --override NOTE`.
18. C3 End state. `s2rebuild step C3` checks that the workspace holds exactly two entries for the run, the live database and the retained copy (see below), that the live name resolves to the recorded live record, and that a write on the retained name is refused; the listing goes into the journal and the final report. It issues no DETACH, ATTACH or DROP; a third entry refuses with the listing. Without the owner profile the block is printed and the gate waits for a person.
19. C4 Remove the migration account. `s2rebuild step C4` drops the runner account; delete its Keychain item. C3 must be passed. The step stops before DROP USER while a view or a routine in the live database has the runner as its definer.
20. C5 Delete the retained original, later and by the customer. `s2rebuild cleanup` lists what the run left: the live record, the retained copy with its record, its size in GiB and the retain-until date. After that date `s2rebuild cleanup --delete <retained>` prints the record and asks you to type the name again; anything else keeps the copy and records "kept by customer". The delete is DETACH, re-attach by the record's own syntax read write, a record check, DROP, each behind its check, then a check that the live name still resolves to the live record. `--yes` does not apply, and no configuration key makes a run delete the retained copy.

After a run: two entries

After a run the workspace holds two databases:

- the live database, under the live name: a branch (ATTACH_TYPE `pitr`) of the replacement, with the target layout, read write, with every table, view, routine and pipeline of the capture;
- the retained copy, under the retained name: a branch (ATTACH_TYPE `resume`) of the original's own storage, attached read only, every row as it was at the freeze.

Why two and not one: the retained copy is the way back and the audit copy. It shares the original's STORAGE_ID but is its own record; the original's own record was attached, proved unchanged against the capture and dropped inside the switch (S2b), and the replacement name was dropped once the live branch stood on its own (S4). So no second outage is needed and nothing is left to delete by hand. The kit never deletes the retained copy. C2 records `retain_until_utc` (the switch time plus `closeout.retain_days`, 90 by default; on the hand route the time of `gate O10 --pass` stands for the switch time) in the journal, the final workbook and `run-report.md`, with the C5 command. After that date the customer runs `s2rebuild cleanup --delete <retained>` and types the name again; the dropped record stays recoverable with `ATTACH DROPPED DATABASE` for the engine's retention period.

What the Portal shows during the window: from O3 the source is listed as attached read only (the Portal may show 0 rows for its tables; the kit's counts are the reference); from O5 the replacement is listed as a new database; after S1 the source shows as detached and after S2 the retained name appears as "Read Only Legacy"; for the seconds of S2b the live name is attached read write on the original storage, then its record leaves the list; from S3 the live name is attached read write on the replacement's storage; after S4 the replacement name leaves the list.

Troubleshooting

- Re-copy after O8: `s2rebuild mark-recopy TABLE [--bucket N]` sets O6, O7 and O8 back to pending and clears the table's verification. Run `s2rebuild run --from O6`; O6 copies the buckets again, then O7 and O8 run again. The switch refuses until O8 passes again. A fresh-table load reopens O7 and O8 the same way. Both rewrite `owner-sql/O10.sql` and `O10-rollback.sql` with a STOP; after O8 passes, run `s2rebuild owner-sql O10` (and `--rollback`) or `s2rebuild owner-notebook` again, and never run the O10 cells of an older notebook. Once the switch may have begun (O10 passed, the go recorded, an O10 gate, or the retained or live attachment recorded), a re-copy is refused; a new copy is a new run.
- Chunks failing during the copy (error 1997 when autoscale or a restart replaces a node, 2013 when a connection drops): each failed chunk is reconciled before it runs again. After `copy.max_consecutive_failures` (default 6) failures in a row the circuit breaker stops new chunks; at the end of the pass the kit retries every chunk left once, the failed ones no sooner than 20 minutes after their failure. When the retry round fails too, O6 ends INCOMPLETE with "chunks are failing in a row; nodes may be restarting, see MV_EVENTS; rerun with --retry-failed when the workspace is steady". Check MV_EVENTS, turn autoscale off for the window, then `s2rebuild run --from O6 --retry-failed`.
- Count mismatch after the copy (exit 3): an interrupted INSERT committed on the server after its bucket was reconciled and copied again. The O6 note names each table and its reconciled buckets: wait until 20 minutes after the interruption, then `s2rebuild mark-recopy TABLE --bucket N` for each bucket and `s2rebuild run --from O6`.
- Workspace not ACTIVE (suspended, resuming, scaling): the kit cannot see the state; answer the three questions of P2 with `--workspace NOTE` at O1, and never start the copy while the workspace scales.
- Name resolution lost (Errno 8) or connection refused: every connect is tried 3 more times, 20 seconds apart, then fails with the host name; flush the Mac's resolver cache and run the step again.
- A node reset the connection (driver error 2003, 2004, 2006 or 2013): the command prints one plain line, "a node reset the connection; the workspace may be scaling or restarting; rerun the step", and exits 2; the raw error is in the run's log file. The step is recorded failed and re-enters from the journal on the rerun. A workspace that suspended during the window shows the same: resume it in the Portal, run `s2rebuild check`, then the step again.
- Auto-suspend: while a step runs, the kit's keep-alive thread sends `SELECT 1` every `workspace.keepalive_minutes` (default 15; 0 disables) on its own runner connection. Between commands nothing runs (the wait for a gate, the time between O8 and O10), so disable auto-suspend for the window, or run a notebook job with the same query (STEPS P4). Auto-suspend counts queries from connections as activity, not pipeline batches.
- Reference tables: an INSERT into a reference table runs on the master aggregator, which a READ ONLY source does not have (error 1777). The copy stages each reference table through a sharded table in the replacement; `tables.csv` marks it `reference` with `copy_path` staging, and the count comes from `SELECT COUNT(*)` because TABLE_STATISTICS reports no rows for it. A reference table that shows 0

rows in the plan means the capture is from a kit before this release: run `s2rebuild profile --redo`.

- Password policy (error 1144 at CREATE USER): the workspace enforces the `PASSWORD_` variables (minimum length, character classes, repeated and sequential characters). `s2rebuild account-sql --execute` reads the policy through the owner session, prints it and generates a runner password that meets it. A hand-run block with a weaker password fails with 1144: use the generated one, or make the password meet the printed policy.
- Portal row counts on a READ ONLY source: the Portal may show 0 rows for the tables of the frozen source (it reads the statistics of the read-write attachment, which the frozen source no longer has). The rows are there. The kit's counts, taken through its own session and recorded in the journal and the O4 capture, are the reference; O3 says so in its log.
- A third entry after the switch: someone attached a database under one of the run's names; C3 refuses with the listing and touches nothing. Do not delete a record in the Portal by STORAGE_ID: the retained copy shares the original's.
- Lock waits (error 1205): the kit serializes the table, probes the lock and retries after `copy.lock_retry_seconds`. A lock held by a killed client clears in 15 to 20 minutes. A chunk fails after `copy.max_lock_waits` lock waits and O6 stops INCOMPLETE with the command to run: `s2rebuild step O6 --fresh-table TABLE` loads the table into a new table and renames it. It refuses while the stop file exists. After a failure, rerun the same command; it resumes and does not load a complete fresh table again.
- Detach waits: a DETACH blocked by writers is cancelled after `freeze.detach_wait_seconds` and issued with KILL when `freeze.detach_kill` is true. With `detach_kill: false` the step stops; stop the writers, then `s2rebuild step O3 --redo`. An idle 3 TB database detached in 73 seconds at O3 and attached read write again in 8 seconds.
- Name collisions: names are not unique on Helios, and a STORAGE_ID names a storage that several branch records share. Every ATTACH names a STORAGE_ID, and every check of the switch, the rollback, C3, C5 and abandon compares the branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID) with the one the journal recorded. `s2rebuild check` refuses a replacement or retained name that already exists.
- Routines missing or not visible: the runner sees no row in `information_schema.ROUTINES`, so P2 prints "routines: not visible to the runner; owner profile needed" without the owner profile, and O5 and O7 end pending with the routine names. Configure the owner profile and run `s2rebuild profile --redo`. A routine an aggregate names and `objects.routines.ignore` lists stops O5 with the aggregate and its error; remove the name from the list.
- Pipelines the application recreates: list them in `databases[].app_managed_pipelines`. They are stopped at O2, never replayed, and C1 reports whether they came back. Pipelines of a source type skipped at the credential prompt stay skipped for the run (the journal records the decision): the plan sheet, the O9 block and the final report list them with the placeholder file `pipelines/recreate-with-placeholders.sql`, from which the customer creates them on the live name after O11 with their own credentials.
- Configuration changed after O1: O1 records the configuration hash. Every command that opens the run journal refuses a configuration that hashes differently, until `--accept-config-change NOTE` records both hashes and the note in the journal. A changed database name is refused with or without it: use the configuration of the run. `s2rebuild status` reports both.
- Abandon a run before the switch: run `s2rebuild pause` and wait until the copy step ends, so no chunk still reads the source. `s2rebuild abandon` prints the block; `s2rebuild abandon --execute --note TEXT` runs it with the owner profile. It attaches the source READWRITE again as its recorded

record, drops the replacement only while its name holds the record O5 recorded and no branch was made from it, and closes the run: `run`, `step` and `gate` refuse it afterwards. Then remove the runner with `s2rebuild account-sql --drop --execute`. A new run starts at P1. Once the switch may have begun, abandon refuses; the way back is `s2rebuild owner-sql O10 --rollback`.

- An admin step stopped at a check: the error names the check, the expected value and the observed value. Fix the cause and rerun the step; the freeze and the switch re-enter from the live attachment state.

Documents

- docs/STEPS.md: every step in full.
- docs/CONFIG.md: every key, flag, variable and exit code.
- docs/COVERAGE.md: the coverage grid, what the kit migrates and what it does not.
- docs/CODE-GUIDE.md: modules, data flow, decisions you can change.
- docs/LESSONS.md: Helios facts and run lessons.
- docs/ARCHITECTURE.md: the design.
- docs/ROADMAP.md: what comes after release 1.0, with effort.
- docs/RELEASE-1.0.md: the release 1.0 work order and the live test ladder.
- `docs/s2rebuild-guide.pdf`: this README and STEPS with the flow diagram, built by `docs/build_pdf.py`.
- `tests/engine/helios-<version>.json`: the record of the engine the test fake is built from, written by `tools/enginecapture.py --fixture-database NAME` with the owner profile (reads only); refresh it per engine version as docs/CODE-GUIDE.md says.

Steps

Author: John Bergamini

One section per step. Each lists the purpose, the identity, inputs, outputs, the statements the kit issues, the gate, how to re-enter and what to do when it fails. Admin steps print their block (also written to `customer/runs/<run>/<db>/owner-sql/<step>.sql`, mode 0600); with the owner profile the kit executes it statement by statement and stops at the first check that does not return the expected value. Every statement passes the step's guard first.

Commands below assume `s2rebuild` on the path and `customer/config.yaml` as the configuration. Add `--db NAME` when several databases are configured.

A step that passed is skipped on a rerun only while every step before it is passed. When an earlier step is reopened (a re-copy reopens O6, O7 and O8), the passed step is refused like a step that never ran. The refusal names the steps to pass first, earliest first, and how: `s2rebuild gate ID --pass` for a step a person records, `s2rebuild run --from ID` for a step the kit runs. Follow them in that order, since `run --from` stops at a gate a person has not recorded.

The same rules hold for every command that runs a step: `run`, `step`, `check` (P2) and `profile` (P3, O4).

1. A step whose earlier steps are not passed is refused for every selected database before the command connects or writes. Gate: the refusal names the steps to pass. A dry run reports it and goes on.
2. `profile` captures a passed step again only with `--redo`, and the log names what it rewrites. `check` reruns a passed P2 without it.
3. `gate` records a step whose earlier steps are not passed, with a warning that names them. Gate: the warning is printed and logged.
4. O1 records the database names and the configuration hash. Every command that opens the run journal refuses changed names. It refuses a changed configuration unless `--accept-config-change NOTE` records an event with both hashes and the note. Gate: `s2rebuild status` shows no warning. A dry run and `status` report a changed hash and go on.
5. Every command works on the run that `--run ID` or `S2REBUILD_RUN` names, else on the most recent run. `run` resumes that run and skips the steps that passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names it; an id that exists is refused). Once a run exists, `run --run ID` refuses an id that names no run. Every gate hint and every log header prints the run id with its `--run` form. Gate: one run directory holds P1 to O11 of the night.

P1 Prerequisites

- Purpose: an account for the copy, its password stored, the configuration written.
- Identity: owner (account) and the operator.
- Inputs: `examples/config.example.yaml`, the admin.
- Outputs: the runner account with the privileges of section 5 of the architecture; a Keychain item or variable per identity; `customer/config.yaml`.
- Statements: `s2rebuild account-sql` prints CREATE USER and GRANT with checks before and after; `--execute` runs them with the owner profile and injects the password at execution only.
- Owner password: with `connections.owner.password_keychain`, `account-sql --execute` asks for the owner password (hidden, never echoed) when the item is missing and stores it with `security add-generic-password -U`; with `password_env` the variable is read as it is.

- Runner password: with `connections.runner.password_keychain`, `account-sql --execute` reads the `PASSWORD_` variables through the owner session, prints the policy, generates a 32 character password that meets it (every character class, no character twice in a row, no three sequential characters; the policy minimum when it asks for more) and stores it in the item before CREATE USER. Nothing is asked and the value is never printed. An account that exists already keeps its item, and the block stops at its first check. With `password_env` the variable is used as it is.
- Gate: the runner account, the Keychain items (or variables) and the configuration are in place. Record: `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`. Once a run exists, `--run ID` must name it.
- Re-enter: rerun `account-sql`; its first check stops when the account already exists.
- When it fails: a missing privilege is named by `check` (P2); grant it and rerun `check`.

P2 Preflight

- Purpose: prove the connection, privileges, layout, storage id, names and compute size before anything else.
- Identity: runner, plus the owner when configured.
- Inputs: the configuration; P1 recorded in the run. Until then `check` refuses and creates nothing.
- Outputs: the source STORAGE_ID in the journal, the engine version, a preflight record.
- Statements: SELECT, USE and SHOW only: CURRENT_USER and version, USER_PRIVILEGES for the runner, DISTRIBUTED_DATABASES, MV_CLOUD_ATTACHED_DATABASES, MV_CLOUD_BRANCHES by the source STORAGE_ID (as the owner when configured), MV_NODES leaf cores; with the owner profile the PIPELINES count, ROUTINES, SHOW AGGREGATES and SHOW CREATE per routine.
- Workspace: P2 prints three questions, "is the workspace ACTIVE?", "is autoscale off for the window?", "is auto-suspend off for the window, or a keep-alive job in place between the commands?", and O1 records the answers from `--workspace NOTE`.
- Gate: every line ok: all runner privileges present, source reported with a STORAGE_ID that holds the source record only (a branch of the source or a detached record on its storage would be a third entry when the switch ends, found only after S2b drops the original; remove it with the customer or leave the database out of the run), replacement and retained names free, leaf cores times 8 at or above the target partitions (when MV_NODES is readable; otherwise confirm in the Portal). The workers line is information, not a gate: `copy.workers` and `copy.table_concurrency` as configured, and the leaf cores read (client concurrency is not the limit, LESSONS fact 42). The routine line prints "N routines (f functions, t TVFs, a aggregates, p procedures), M ignored by configuration" with the owner profile; without it, "routines: not visible to the runner; owner profile needed", never "none", since the runner sees no routine.
- Re-enter: `s2rebuild check` any time; a passed P2 runs again without `--redo` and records its result again. O1 needs P2 passed in the same run.
- When it fails: each failed line says what to change (grant, rename, compute size).

P3 Profile

- Purpose: know the database: tables, keys, sizes, accounts, sessions, views, routines, pipelines; estimate the copy.
- Identity: runner; owner for the pipelines, the routines and the write-capable accounts (the runner sees no routine and no other account's grants).

- Inputs: the configuration; P1 and P2 passed in the run.
- Outputs: `inventory/P3/inventory.json`, `tables.csv` (with `storage_type`, `reference` and `copy_path` per table), `definitions/` with `definitions/routines/<name>.<kind>.sql` per routine (kind function, tvf, aggregate or procedure), `pipelines/` (0600, CREDENTIALS replaced by `<<CREDENTIALS>>`), `reports/profile.xlsx` with the estimate.
- Statements: SELECT, USE and SHOW only (SHOW CREATE PIPELINE ... EXTENDED, ROUTINES, SHOW AGGREGATES and SHOW CREATE FUNCTION, AGGREGATE or PROCEDURE with the owner profile). A reference table (`CREATE [ROWSTORE] REFERENCE TABLE`) is always counted with `SELECT COUNT(*)`, because `TABLE_STATISTICS` reports no rows for it; a count that returns no row refuses the capture by table name. Its plan is one bucket on the staging path; the table tab says "reference table, copied through a staging table".
- Pipeline credentials: on route `kit`, when the capture holds pipelines with a CREDENTIALS clause, the kit asks for the credentials of each source type that has none yet, by field: S3 access key ID (shown), secret access key (hidden), session token (hidden, Return to skip); Kafka SASL user (shown) and password (hidden); for another source type it prints the CONFIG keys the pipelines use and asks for credential keys and values until a blank key. It assembles the CREDENTIALS JSON and stores it in the Keychain item named in `pipelines.credentials_keychain` for that type (a type without an item is not asked; export its `pipelines.credentials_env` variable at O9 instead). A blank first answer skips the source type: the journal records it (`pipeline_skip`), the plan sheet and the O9 block list the pipelines that will not be recreated, the O9 count gate leaves them out, and the final report names them with the placeholder file. No prompt without pipelines, on route `customer`, or without a terminal (O9 asks again). Nothing typed is printed, logged or written to a file.
- Estimate: one line, the rows of the plan divided by `estimate.rows_per_hour` (default 2.4 B, what I5 measured over a whole O6 on an S-8 at 4X). Set the figure your own run measured; the final report prints the actual rows per hour. No step stops on the estimate.
- Gate: no pipeline loads into a procedure that `objects.routines.ignore` lists or the capture lacks (O9 would find no target). The plan sheet and the workbook Summary print the routines by kind and the ones ignored by configuration; without the owner profile they print "not captured".
- Re-enter: once P3 passed, `s2rebuild profile --redo` captures again; the log names what it rewrites. `--exact-counts` for COUNT(*) per table. `--dry-run` writes only `reports/profile-dry-run.xlsx` (or `--out`), never the capture under `inventory/`.
- A stop part way (a lost connection, Ctrl-C) leaves P3 not passed; a plain `s2rebuild profile` captures again.
- When it fails: `P3 refused` exits 1, records P3 failed and writes no workbook; the refused capture stays under `inventory/P3/` for review and the plan sheet. It names each pipeline whose load target is a procedure the replacement will not hold; remove the name from `objects.routines.ignore` (or list the pipeline as application managed) and run `s2rebuild profile` again. A pipeline with an empty definition, or a CREDENTIALS clause the kit cannot replace, refuses the capture by name and nothing is written; run `s2rebuild profile` again.

P4 Plan

- Purpose: the decisions people make before the night.
- Identity: the people named in the plan.
- Inputs: `s2rebuild plan`.
- Outputs: a recorded gate.

- Statements: none.
- Gate: the window (when configured), the named owner for "application stopped", the pipeline credential route and handover, the retained name and retention days, recorded in the configuration; autoscale off for the window; the auto-suspend choice below.
- Auto-suspend between commands: while a step runs, the kit's keep-alive thread sends `SELECT 1` on its own runner connection every `workspace.keepalive_minutes` (default 15; 0 disables), so a long statement or a wait inside a step never looks idle. Between two commands nothing runs (the wait for a gate, the time between O8 and O10). Choose one for the window and record it in the P4 note: disable auto-suspend for the window in the Portal (the default choice), or a notebook job of the customer that runs `SELECT 1` every 15 minutes through the window. The plan sheet prints the three workspace questions O1 records the answers to.
- Re-enter: edit the configuration and print the plan again.
- Before the window: start every `run` and `step` on a Mac on power. On macOS the kit starts `caffeinate -i -w <pid>` for each `run` and `step`, so the Mac does not sleep while the command runs; a sleeping Mac is the likeliest real interruption of a copy (Helios fact 12). Between commands nothing keeps the Mac awake; run `caffeinate -i` in a terminal for the night.

O1 Start

- Purpose: start the window with a recorded go decision.
- Identity: runner and the operator.
- Inputs: the window in the configuration, when there is one; P2 passed in this run.
- Outputs: names, window and the configuration hash in the journal. The window is information: no step stops on it. Without a window O1 records an empty one.
- Names: `run.mode` is `production` (the default) or `test`; `--mode` overrides it. In production, when `databases[].replacement` or `retained` is not set in the configuration, O1 asks for each name after its checks passed, with the kit's date default as the suggestion (Return keeps it), checks that it is an identifier unused by the run, records the answers in the journal (a `names` event and the O1 note) and prints both names before the go. Every later command reads the typed names back from the journal. In test mode the defaults apply silently. Without a terminal, production mode refuses and says to set the names, type them at a terminal, or run with `--mode test`.
- Several databases: `s2rebuild run` runs O1 for every selected database, then stops once for the go. `s2rebuild gate O1 --pass` records the go in every journal. A refused or failed O1 stops the run at once.
- Statements: none beyond reading the capture.
- Refusal: without P2 passed in this run, O1 refuses before it records the window, the names or the configuration hash; its note says why. Run `s2rebuild check` in this run.
- Workspace: O1 prints the three questions of P2 and records the operator's answers from `s2rebuild run --from O1 --workspace NOTE` as evidence of the go (an event and the O1 note); without the flag the answer line says none is recorded. Autoscale must be off for the window: a node replaced under a chunk fails it.
- Gate: P2 passed in this run, go recorded: `s2rebuild gate O1 --pass --note TEXT`. The note records whether the application stays stopped until O11. Only a stopped application keeps the O10 rollback possible after S3.
- From O1 on: a changed configuration is refused by every command until `--accept-config-change NOTE` records it; a changed database name is refused with or without it.

- A go recorded in a journal where O1 never ran is recorded with a warning: the checks and records of O1 are missing there. Run `s2rebuild run --from O1 --to O1 --redo`, then record the go again.
- Re-enter: `s2rebuild run` resumes the same run. It skips O1 once the go is recorded and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.

O2 Stop pipelines

- Purpose: stop every pipeline and record the final bookmark of each.
- Identity: owner.
- What a pipeline and an offset are: a pipeline is a server-side ingestion job reading a source (a Kafka topic, an S3 prefix and others) into one table. Its offset is its bookmark per source partition. A pipeline recreated with its recorded offsets resumes where the old one stopped: no message skipped, none loaded twice.
- Outputs: `inventory/O2/` with every definition (credentials replaced by the placeholder) and the offsets read after the stop; journal pipeline rows.
- Statements: STOP PIPELINE per pipeline (skipped only for one already Stopped, so a pipeline in state Error is stopped too), a check that none is Running, then the capture. The stop list is the earlier capture plus any pipeline the live view shows.
- Gate: no pipeline Running; every captured pipeline Stopped; LATEST_LOADED_OFFSET equals LATEST_SUCCESSFUL_OFFSET on every partition, since O9 replays the committed position. A pipeline that runs again before the capture refuses the gate. A partition where the two offsets differ refuses it too, a NULL successful offset beside a loaded one included, with the partitions listed; rerun O2 once they agree. A pipeline whose SHOW CREATE PIPELINE returns no CREATE statement, or whose CREDENTIALS clause has a form the kit cannot replace, refuses the capture by name and nothing is stored; rerun O2. A topic, table or column named credentials does not refuse it.
- Notebook route: the O2 cell's result carries the capture; `s2rebuild gate O2 --pass --note TEXT --evidence FILE` imports it.

O3 Freeze

- Purpose: make the source read only for every account.
- Identity: owner; the write probe runs as the runner.
- The freeze: checks (READWRITE, recorded STORAGE_ID, no pipeline Running), DETACH, check detached, ATTACH ... READ ONLY STORAGEID ..., checks (READONLY, same STORAGE_ID), then the write probe: a DELETE for a value that cannot exist on a string column (or an integer key below its minimum). Pass means the server refused it as read only. Then freeze_time and, with the owner, a grants reference for C1.
- DETACH waits: after `freeze.detach_wait_seconds` the kit cancels the waiting DETACH on its own connection and issues DETACH ... KILL when `freeze.detach_kill` is true; otherwise it stops: stop the writers, then rerun the step.
- Gate: probe refused and READONLY.
- After the probe the log says that the Portal may show 0 rows for the tables of the frozen source (fact 38); the kit's counts in the journal and the O4 capture are the reference. The step line reads `[O3] Freeze (admin; owner)`: the redactor masks the owner and runner user names only where they stand as a user, never in a label.
- Re-enter: `s2rebuild step O3 --redo`. When the source is already READONLY on the recorded STORAGE_ID the kit only checks, probes and records freeze_time, with or without the owner profile.

- Re-enter between DETACH and ATTACH: when no database carries the source name and the recorded source record (name, STORAGE_ID, COMPUTE_ID; never the first row of its storage) shows IS_DETACHED = 1, `s2rebuild step O3 --redo` prints the re-entry block to `owner-sql/O3-reentry.sql` and runs it with the owner profile: checks (storage detached, no database named source or replacement), ATTACH ... STORAGEID ... READ ONLY, the READONLY checks, then the probe and freeze_time. It never detaches again. The kit reads this position with the owner session when the owner profile is configured, otherwise as the runner. Without the owner profile, run the printed block, then `s2rebuild step O3 --redo`. The notebook O3 cell runs the same block (`O3-reentry`) when the whole block stops at its first check with no rows. A database under the source name on another STORAGE_ID still refuses at the first checks.
- Hand route: `s2rebuild owner-sql O3` writes and prints both blocks, `owner-sql/O3.sql` and `owner-sql/O3-reentry.sql`. The whole block opens with a note that says when to run the re-entry block instead: its first check finds no rows. Gate: the re-entry block's checks pass; a database under the source name stops it before the ATTACH.

O4 Freeze capture

- Purpose: the inventory the copy uses, taken after the freeze.
- Identity: runner.
- Inputs: `s2rebuild run` takes the capture itself after the O3 gate (the runner session reads; the owner session, when configured, reads the routines and the write-capable accounts); `s2rebuild profile --step O4` stays for a rerun (`--redo`), for `--exact-counts` and for `--all-databases`. Both refuse until O1 to O3 passed, before they connect or write. A capture already present is checked and kept, never taken again without `--redo`.
- Outputs: `inventory/O4/`, `drift.json` against P3, the copy plan. Reference tables are counted exactly again, as at P3, so the plan carries the frozen count.
- Gate: drift reviewed (a routine added, removed or changed since P3 is listed by kind and name; a pipeline added, removed or whose definition changed is listed by name; states and offsets are not drift); the load-target rule of P3 holds again, so a pipeline into an ignored procedure refuses O4 and records it failed; O5 and later refuse until O4 passes.
- Pipelines: the source is READ ONLY, where `PIPELINES` is empty (fact 23). The capture reads no pipeline view and records "pipelines not listed: PIPELINES is empty on a READ ONLY attachment; O9 replays the O2 capture taken after the stop"; the drift report prints that note once instead of every pipeline removed.
- Re-enter: once O4 passed, `s2rebuild profile --step O4 --redo` captures again and rewrites the copy plan; the log says so. O5 and O6 refuse a plan that differs from the journal. A stop part way leaves O4 not passed; a plain rerun captures again. A dry run writes nothing under `inventory/`.

O5 Create replacement

- Purpose: the replacement database with the target layout and identical table definitions, then the functions, table-valued functions and aggregates of the capture.
- Identity: runner for the database and the tables; owner for the routines.
- Statements: CREATE DATABASE ... PARTITIONS ... SUB_PARTITIONS, CREATE TABLE ... LIKE per table, SHOW CREATE TABLE (runner); USE, CREATE FUNCTION and CREATE AGGREGATE naming the replacement, with the ROUTINES and SHOW AGGREGATES reads and SHOW CREATE per routine (owner).

- Routines: after the tables, the owner session creates the functions and TVFs, then the aggregates (an aggregate names its functions), each skipped when it exists and checked after; a routine that fails is tried again after the others, in rounds, and a round that creates none refuses the step with each routine and its error number. The DEFINER rule of the views applies: the original definer is kept while that account exists and is not the runner, else the owner; on error 1227 the routine is created again without the clause and the reason is recorded per routine. A routine body that names the source database refuses the step before any statement. A routine `objects.routines.ignore` lists is left behind and named in the log.
- Outputs: the replacement STORAGE_ID in the journal (the switch attaches it by that id); one `routine` event per routine with the definer choice.
- Gate: layout and durability verified; every table definition equals the capture (only the AUTO_INCREMENT counter may differ); every expected function, TVF and aggregate exists in the replacement with the captured definition (normalized, DEFINER aside).
- When it fails: `O5 stopped` names each routine still missing with its error; fix the cause (a function an aggregate names is ignored by configuration, say) and rerun O5; routines already created are skipped.

O6 Copy

- Purpose: copy every row.
- Identity: runner.
- Statements: INSERT INTO replacement SELECT FROM source WHERE CRC32(key) MOD B = k per bucket; reconciliation DELETE of a started or failed bucket before it is re-inserted.
- Behaviour: a reference table (marked `reference` in the capture, one bucket, `copy_path` staging) copies through a sharded staging table in the replacement (`<t>__stage`, created from its definition without keys, filled from the source, copied into the reference table, dropped), because an INSERT into a reference table cannot read a READ ONLY source; its count proof compares the exact count the capture took; a queue per table over `copy.workers` connections (1 to 32); above 8 workers no table takes more than half of them (`copy.table_concurrency: auto`); a table that hits a lock wait (1205) is serialized and probed; an integer `copy.table_concurrency` (1 to `copy.workers`) caps every table at that many chunks in flight instead, with no serialization after a lock wait; the journal is written before each statement; `s2rebuild pause` stops new chunks while running ones finish. A chunk fails at its `copy.max_lock_waits`-th lock wait, and the O6 note names `s2rebuild step O6 --fresh-table` TABLE. Ctrl-C stops new chunks; a second Ctrl-C only prints that a hard stop needs kill, and the workers still finish their chunks. Never hard-stop the copy: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes.
- Failures, the breaker and the retry round: a failed chunk (error 1997 when a node is replaced under it, 2013 when its connection drops) is marked failed, so its rows are reconciled before it runs again, and it queues last in its table. After `copy.max_consecutive_failures` (default 6) chunk failures with no success between them (lock waits not counted), the circuit breaker stops new chunks and the running ones finish (journal event `breaker`). At the end of the pass, unless the operator stopped it, one retry round runs every chunk left; the chunks that failed in the pass start no sooner than 20 minutes after their failure, since a client cut off mid INSERT leaves a transaction that may still commit (Helios fact 12; journal event `retry_round`). A chunk that fails in the retry round is not retried again, and a breaker there ends O6 INCOMPLETE (exit 2) with "chunks are failing in a row; nodes may be restarting, see MV_EVENTS; rerun with --retry-failed when the workspace is steady". A chunk stopped by its lock-wait cap is not retried; its way out is the fresh-table load.
- Connections: every connection has TCP keep-alive on its socket (first probe after 60 seconds idle, then every 30), so a NAT or firewall does not close the connection of an INSERT that runs for hours. A connect

that fails on name resolution or a refused connection is tried 3 more times, 20 seconds apart, then fails with the host name.

- Progress: every `copy.progress_seconds` one line with the tables and buckets done, the rows per hour, the projected finish, and `slow=table/bucket minutes` for every chunk running longer than `copy.slow_chunk_minutes` (default 30). A connection reset of the step itself (driver error 2003, 2004, 2006 or 2013) ends the command with one plain line, "a node reset the connection; the workspace may be scaling or restarting; rerun the step", exit 2, the raw error in the log file.
- Gate: every chunk done; per-table count proof all match.
- Count mismatch (exit 3): the O6 note and the log name, for each table whose counts differ, the buckets that had a reconcile in this run: an interrupted INSERT may have committed after its reconcile, so the bucket holds its rows twice. Wait until 20 minutes after the interruption, then `s2rebuild mark-recopy TABLE --bucket N` for each bucket named and `s2rebuild run --from O6`.
- Re-enter: `s2rebuild run --from O6` resumes; `--retry-failed` reruns parked chunks; `s2rebuild step O6 --fresh-table TABLE` loads a locked table into a new table and renames it. The fresh-table load refuses while the stop file exists, and when the fresh name would be over 64 characters or a planned table. It reuses only a fresh table its journal recorded; another table under that name refuses, and a missing replacement table is never recreated from the source. After a failure, rerun the same command: it resumes from the recorded fresh table and does not load it again once it is complete. After a re-copy mark, or a copy run that finished the table, it loads the table again. The fresh-table load replaces the whole table, so before its first statement it clears the table's verification and sets O7 and O8 back to pending when they passed, and the O10 block files get a STOP as for a re-copy. After it, run `s2rebuild run --from O6`: O7 sets the counters of the new table, O8 verifies it.
- Re-copy: `s2rebuild mark-recopy TABLE [--bucket N]` marks buckets for the next copy run, which deletes and re-inserts them. In the same journal write it sets O6, O7 and O8 back to pending when they passed, and clears the table's `verified_at_utc` and `target_count`. With several databases it checks every selected journal first, so a refusal for one marks none. Just before that journal write it rewrites `owner-sql/O10.sql` and `owner-sql/O10-rollback.sql`, when they exist, so each starts with a STOP, and it says that a notebook generated before must not run its O10 cells. An O7 or O8 re-copy mark and a fresh-table load do the same. Then:
  1. Run `s2rebuild run --from O6`. O6 copies the marked buckets again. Gate: O6 passed.
  2. O7 runs again: counters after the re-copy, then the stale check. Gate: O7 passed.
  3. O8 runs again on the re-copied rows. Gate: O8 passed. Until then S0 refuses the switch and the rollback block stops.
  4. Before a hand or notebook switch, generate the O10 blocks again: `s2rebuild owner-sql O10` and `s2rebuild owner-sql O10 --rollback`, or `s2rebuild owner-notebook`, and use only the new notebook. Gate: the forward block holds no STOP.
- Re-copy after the switch began: refused. Once O10 passed, the go decision is recorded, an O10 gate is recorded (pass or fail), a refused `gate O10 --pass` recorded the switch its evidence shows, or the switch time or the retained or live attachment is recorded, `mark-recopy` and O6, O7 and O8 refuse with nothing written. A passed O6 to O8 is still skipped, so `s2rebuild run` continues to O11. A new copy is a new run.
- An O10 that stopped before the go decision issued nothing, so it does not block a re-copy: a Ctrl-C at the typed-name prompt, a lost owner connection, a stop during S0. A refusal at S0 or at an S1 before-check does not block it either: those checks run before the go. When O10 reads a switch already under way (a hand S1, say), it records that before the go, and the re-copy stays refused. After a hand or notebook switch, record it with `s2rebuild gate O10 --pass` or `--fail`, so the journal knows.

- While O10 runs, it holds the copy lock of the journal. `mark-recopy`, O6 and an O7 re-copy round in another terminal refuse until O10 ends, with nothing marked or copied; the refusal names the process. As O10 records the go decision, it reads the journal part of S0 again in the same journal write. A re-copy mark (an O8 mismatch, say) or O6 to O8 started in another terminal while the operator typed the live name refuses the go, with nothing issued. Then run `s2rebuild run --from O6`, and O10 again once O8 passed.

O7 Post-copy

- Purpose: views, then procedures, counters and the stale check, then the projections.
- Identity: runner; owner for the views and the procedures. A view or a procedure runs under its definer and C4 removes the runner, so the runner never creates one.
- Statements: USE, CREATE VIEW, CREATE PROCEDURE and CREATE PROJECTION naming the replacement (owner), ALTER TABLE ... AUTO_INCREMENT and AGGREGATOR SYNC AUTO_INCREMENT ... ALL, counts and newest values on both sides.
- Source names: a view or procedure definition that names the source database refuses the step before any view statement. Edit the definition in the capture so it names no database, then rerun O7.
- Procedures: once every view exists, the owner session creates the captured procedures as O5 created the functions (rounds, the DEFINER rule, the 1227 fallback recorded per procedure), so a procedure that reads a view compiles and a pipeline INTO PROCEDURE at O9 finds its target. The gate compares each definition with the capture and records the ROUTINES row count of the replacement, which the switch expects on the live name after S3. With no procedure to create (none captured, or every one ignored), O7 still records that count through the owner session, so S3 counts the functions, TVFs and aggregates O5 created. Without the owner profile the procedures wait for the views, as the step ends pending.
- Projections: last, once the stale check and the counters are done, so each one is built once over its full table. The owner session runs USE of the replacement and the captured CREATE PROJECTION as it is (the table resolves against the current database). The gate compares each definition with the capture and records the PROJECTIONS count, which S3 expects on the live name. A captured definition whose FROM names a database refuses O7 before any statement.
- Views with the owner profile:
  1. The owner session creates each missing view. It keeps the original DEFINER when that account still exists and is not the runner. Otherwise it removes the clause and the owner account becomes the definer. When the server refuses the original DEFINER (error 1227: Helios does not let even the admin name another account), the owner creates the view again without the clause. The journal and the Views tab of the final workbook record the choice per view.
  2. A view that fails is tried again after the others, in rounds, so a view on another view is created once the other exists. Gate: every view created. A round that creates none refuses the step with each view and its error number; fix the cause and rerun O7. Views already created are skipped.
- Views without the owner profile:
  1. O7 prints the O7 block and writes it to `owner-sql/O7.sql`: every view in dependency order, each skipped when it exists and checked after. The admin who runs it becomes the definer.
  2. The stale check and the counters run. O7 ends pending.
  3. Run the block as the admin, then `s2rebuild step O7`. Gate: the rerun finds every view and passes.
- Counters: each AUTO_INCREMENT counter is set to the larger of the source counter and MAX + 1, then synced on every aggregator (ALL). A sharded table whose counter the server refuses to set (error 1749) gets the sync alone, which moves every aggregator past MAX. An empty table without a source counter

keeps the default. The table tab of the final workbook shows both values and the decision. A rerun of O7 sets and syncs every counter again.

- Re-copy: a stale mismatch marks the table for re-copy, the kit copies the marked buckets again and repeats the check, for up to three rounds. A round that ends with every chunk done and the count proof all matching marks O6 passed again. The counters are set once, after the last round.
- Gate: views and procedures created, counters set, zero stale mismatches after re-copy.
- When it stops: a re-copy round that does not complete leaves O6 pending; run `s2rebuild run --from O6`, which finishes the marked buckets and reruns O7.

O8 Verify

- Purpose: prove the copy.
- Identity: runner.
- Tiers: 1 counts both sides; 2 newest time value and key range; 3 grouped fingerprints (key hash and value hash per group, mapped to buckets; the value hash takes the time column plus up to `verify.value_columns` supported columns).
- Tier 3 runs on `verify.sessions` connections (default `copy.workers`), largest table first, the source and replacement sides of a table at the same time; one log line per table with the elapsed seconds.
- A table with more rows than `verify.full_fingerprint_max_rows` (default 1,000,000,000) is sampled: a `verify.sample_percent` (default 5) range on its first sort key column or its time column, MIN and MAX from the source, the same predicate on both sides. A table over the threshold with neither column gets a full fingerprint with a note. The verification row and the table tab show the predicate and the rows in the sample.
- `verify.fingerprint_budget_minutes` (default none) is an elapsed deadline: a table whose turn comes after it is recorded not fingerprinted.
- Outputs: verification rows, each with the table's re-copy epoch and the sample predicate, `verified_at_utc` per table, the final workbook and run report.
- Gate: tiers 1 and 2 pass everywhere; tier 3 a full match on every table, or a sampled or not fingerprinted result accepted with `s2rebuild step O8 --yes`. With `--yes` and no chunk copied since the last O8 run, the recorded result is accepted without a rerun and an `accepted` event names each table; otherwise the tiers run again first.
- Rerun one table: `s2rebuild step O8 --fingerprint TABLE --percent N` (sampled) or `s2rebuild step O8 --full TABLE` runs tier 3 for the named tables only and records the result at the current epoch; the other tables keep their recorded result. Add `--redo` when O8 already passed.
- When it fails: exit 3; the mismatching buckets are marked for re-copy, which reopens O6 and O7. Run `s2rebuild run --from O6`: O6 copies them again, O7 resets the counters, O8 verifies again.

O9 Pipelines

- Purpose: every pipeline exists in the replacement, stopped, at the offsets recorded at O2. O9 runs after O7, so a pipeline INTO PROCEDURE finds the procedure O7 created; P3 and O4 refuse a load target whose procedure is ignored by configuration or absent from the capture.
- Identity: owner.
- Credentials: the server hides CREDENTIALS from every identity, so the customer supplies them. Route kit: the kit reads each pipeline's credentials from the Keychain item named in `pipelines.per_pipeline_keychain` or `pipelines.credentials_keychain` (filled at the P3

prompt), else from the variable named in `pipelines.per_pipeline_env` or `pipelines.credentials_env`, exported for this step only and unset afterwards; a source type still without credentials is asked for at O9 as at P3, and a blank answer skips it (its pipelines are left out of the count gate and listed in the block, the log and the final report). The kit injects the value as a SQL literal at execution and masks it in every log line. In the notebook the cell asks for each value with a hidden prompt. Route customer: the customer runs `pipelines/recreate-with-placeholders.sql` from the O2 capture inside the replacement, filling in CREDENTIALS; the kit runs only the checks.

- Statements: USE replacement, CREATE PIPELINE per pipeline (skipped when it exists), its ALTER PIPELINE ... SET OFFSETS statements. Never START.
- Gate: count equals the capture minus application-managed pipelines; none Running; the committed offset of every partition in the replacement equals the O2 capture. The committed offset is LATEST_SUCCESSFUL_OFFSET; O2 refuses a capture that holds a loaded offset without a successful one. The block checks each partition, so the notebook and hand routes hold the same gate.
- Refusals: only the O2 capture is replayed (an earlier capture's offsets were still moving); a definition that names the source database is refused. A definition that still carries a redaction marker (a value the server masked) is refused by name before any statement, by the kit and by the notebook cell; capture it again.

O10 Switch

- Purpose: the live name resolves to the replacement, the original stays readable, and the workspace ends with two entries: the live database and the retained copy.
- Identity: owner.
- S0 (decision): O8 and O9 passed; every planned table verified after freeze_time and after its last chunk, at its current re-copy epoch (a table without a stamp refuses, by name); both storage ids recorded; the operator types the live name (or passes `--yes`). The go is recorded only while the journal part of S0 still holds, and O10 holds the copy lock from S0 to S4 (O6 Copy).
- S0 on every route: without the owner profile O10 refuses on the journal part of S0 (O8 and every step before it passed, freeze_time, every table verified) and prints the forward block with a STOP first. `s2rebuild owner-sql O10` and the notebook O10 cell carry the same STOP while it holds, and `s2rebuild gate O10 --pass` refuses. Generate the forward block or the notebook after O8 passed, and again after a re-copy (O6 Re-copy, step 4).
- A forward block generated before a re-copy: the kit rewrites the files under `owner-sql/` with a STOP, but cannot reach a notebook already uploaded. When `gate O10 --pass` refuses and its `--evidence` shows the switch ran (the retained and live attachments), the kit records those attachments and the pass stays unrecorded. From then on a re-copy is refused. The refusal names the rollback block. Its proof needs the copied row counts; when a re-copy cleared them, the block stops and the way back is a reconciliation. Without evidence, record what ran with `s2rebuild gate O10 --fail --note TEXT --evidence FILE`.
- When S0 refuses: the refusal names each table and the way to verify again. `s2rebuild step O8 --redo` when O7 passed after the last chunk; `s2rebuild run --from O6` while O7 is pending after a re-copy.
- S1: checks (source READONLY on its recorded STORAGE_ID and its recorded record, no foreign active session, replacement READWRITE as its recorded record, retained name free, each storage holding its recorded record only (a branch of the source or of the replacement shares its STORAGE_ID, so the switch refuses here with the records named, not after the DROP of S2b)), DETACH (with the wait and KILL rule), checks: the name free, and the original's detached record with its ATTACH_SYNTAX (the AT MILESTONE form), which the kit records for the rollback. The check reads the original by its record, never

the first row of its storage: a branch of the source or a dropped record of an earlier attempt shares the STORAGE_ID. The kit runs the before-checks once before the go and again before the DETACH.

- S2: ATTACH source AS retained READ ONLY STORAGEID ..., checks (READONLY, a resume record of the original, recorded), one small table count.
- S2b (the live name is free): the original record's own ATTACH_SYNTAX (the AT MILESTONE form S1 recorded) attaches the original read write under the live name; checks: the recorded record, attached READWRITE; the zero-change proof: the three smallest non-empty tables hold the journal's source counts and the S1 session query shows no foreign non-idle session; then DROP DATABASE of the original record; checks: the record dropped, the name free, the original storage holds one entry (the retained record), one small table count in the retained copy. A failed proof stops with the original attached read write under the live name, the rollback state: reconcile, then `s2rebuild step O10` continues at the proof, or stay on the original. For those seconds the live name resolves to the original; the O1 decision that the application is stopped and the session check cover it.
- S3: checks, ATTACH replacement AS live STORAGEID ..., checks: READWRITE, not the original storage, a pitr record of the replacement (recorded), target layout, table, view, pipeline and routine counts (the routine count is the ROUTINES row count the O7 gate recorded), three smallest tables against the journal.
- S4: checks: the replacement name holds the recorded replacement record (create, the parent of the live record) and the live name is its own branch of it; DROP DATABASE of the replacement; checks: the record dropped, the name free, the live record unchanged, one small table count on the live name.
- Gate: the entries with IS_DROPPED = 0 on both storages are exactly the live record and the retained record (two entries). The listing before S1 and after S4 goes into the journal (`switch_listing`). A failed gate names the extra or missing entries.
- The records, in plain words: before the switch the source is one record (`create`) on its storage and the replacement one record (`create`) on another. S2 adds a `resume` record of the source's storage under the retained name; S2b removes the source's own record; S3 adds a `pitr` record of the replacement's storage under the live name; S4 removes the replacement's record. Each ATTACH names a STORAGE_ID, and each check compares the whole record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID) with the one the journal recorded, since the retained copy shares the original's STORAGE_ID and the live database the replacement's. The journal `attachments` table holds the four roles (source, replacement, retained, live) with their ATTACH_SYNTAX; the final report prints them.
- Re-enter: the position comes from the attachment views. A rerun repeats the after-checks of the step it lands behind, so a failed check never turns into a passed switch.
- Write boundary: S3 attaches the live name READWRITE, so the application can write from S3, before O11.
- Rollback (before O11 only), by record: the zero-change proof when S3 ran, R1 detach the live name while it holds the recorded live record, then one of two legs. Before S2b ran: R2a, the original record's own ATTACH_SYNTAX (the AT MILESTONE form S1 recorded; the kit reads it from the live record and refuses when it differs from the recorded one); the retained copy stays attached read only. After S2b the original record is dropped, so the rollback goes through the retained copy: R2b, `ATTACH DATABASE <retained> AS <source> STORAGEID <original>`, a read-write pitr branch of the retained copy under the live name (I0b proved it, Helios fact 35). No rollback step drops the retained copy. The rollback resumes from any position behind S1; behind S3 it starts at R2 without the proof. Every statement runs behind a record check: a record that differs refuses with both records printed and nothing issued. After a rollback through the retained copy the switch cannot continue; a new switch is a new freeze and copy.

- Zero-change proof: every planned table on the live name holds the row count of the copy, no pipeline is Running on it, and no foreign non-idle session is active on it. It detects changed row counts and active writers only, not an update that keeps the count. After S3 a rollback is safe only when the O1 note records that the application stayed stopped.
- Rollback blocks: `s2rebuild owner-sql O10 --rollback` prints two, `O10-rollback.sql` (R1, R2a: before S2b ran) and `O10-rollback-retained.sql` (R1, R2b: after S2b ran), and the notebook holds both. Use the one the S2b state calls for; the R2 before check of the wrong one stops it. Both carry the proof as checks before R1. Generate them or the notebook (`s2rebuild owner-notebook`) after O8 passed, so they hold the copied row counts. Generate them again just before a rollback, so they hold the O11 state and, after S1, the recorded ATTACH_SYNTAX of the original as the R2a statement; before S1 the block carries a milestone placeholder there and says so.
- Behind S3 the notebook cell stops at the first proof check, with nothing issued. Finish by hand from the R2a or R2b check of the printed block.
- Rollback refused, before any statement: once O11 started releasing writers, once C1 saw application writes, or when the proof fails. The kit knows that O11 started from release_started_utc, released_utc or the O11 status. After a hand or notebook O11 the journal may not know, so the proof refuses on the live state: a pipeline Running. A failed proof lists what differs and says reconcile first: a new freeze and a reconciliation, never R1.
- Helios keeps the replacement's STORAGE_ID for the live branch. S3 then passes only when the live name has its own branch row (PREV_DATABASE_NAME the replacement, not dropped or detached) and a COMPUTE_ID other than the replacement's. A rerun of `s2rebuild step O10` at any position issues nothing it already did and reruns the after-checks of the step it lands behind. Identity everywhere is the branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), never a STORAGE_ID alone: the retained copy shares the original's STORAGE_ID and the live branch the replacement's.

O11 Release

- Purpose: writers back on the live name.
- Identity: owner and the customer.
- Retention check: for every captured partition the recorded committed offset must not be below the current EARLIEST_OFFSET, and the partition must exist. A gap means messages were lost to the source's retention; decide EARLIEST or LATEST per pipeline and record it in the gate note.
- Statements: START PIPELINE per expected pipeline (skipped when Running), SET GLOBAL query_parallelism_per_leaf_core when configured.
- Release start: with the owner profile the kit records release_started_utc in the owner session, once the retention check and every check ahead of the first statement passed, just before that statement. From then on a rollback is refused, even when O11 stops part way. An O11 that stops before its first statement (a failed check, a lost connection) records nothing, and the rollback stays open. A hand or notebook O11 records nothing; there the rollback proof refuses once a pipeline runs on the live name.
- Gate: every expected pipeline Running; the application restarted and writing. The kit stamps released_utc when every check passed; recording the gate by hand stamps it too.
- Re-enter: `s2rebuild step O11` after a failure. A pipeline already Running is skipped, so no START is issued twice.
- Pipelines left stopped: with `pipelines.start_at_release: none` O11 starts no pipeline. It checks that every captured partition is present and that none is Running, and notes the START PIPELINE form for later. C1 then reports the pipelines as left stopped and takes no offset sample.

C1 Verify after release

- Identity: runner; owner for pipelines and grants.
- Checks: a writer sample (three PROCESSLIST samples of non-idle sessions from other accounts on the live database; it needs PROCESS), count changes on the largest tables (two samples a minute apart and against the copy), pipelines Running with offsets advancing where data waits, application-managed pipelines present again, no pipeline errors since the release, grants equal to the reference O3 recorded. Application writes are recorded; from then on a rollback is a new freeze and a reconciliation.
- Gate: every check ok and application writes seen on the live database (a session, counts moving, or counts that differ from the copy). A live database that is idle on purpose passes with `s2rebuild step C1 --idle-workload NOTE`; the note is journaled. Without writes or the flag C1 fails with exit 4. A missing application-managed pipeline fails C1. Without the owner profile the pipeline and grants checks are recorded as not checked, no owner profile, and C1 passes with that note.
- Re-enter: `s2rebuild step C1`. It samples again; writes made earlier still show as counts that differ from the copy, and writes an earlier C1 attempt of the run saw (journal `app_writes_seen`) still count when the new sample sees none, with the time named in the note.
- When it fails: fix what the note names and rerun. When the kit cannot see the evidence, record C1 by hand with `s2rebuild gate C1 --pass --note TEXT --override NOTE`. Gate refuses C1 without `--override`; the override is journaled.

C2 Report

- Identity: runner.
- Outputs: `reports/final.xlsx`, `reports/run-report.md`. The Summary sets the copy time the estimate expected against the actual one and prints the actual rows per hour, the figure for `estimate.rows_per_hour` on the next run.
- What the final workbook and the run report carry, from the journal: the latest verification result per tier and table on the Summary (the history stays on the table tab) with the sampled fingerprints and their predicate; the reference tables; the two entries the run leaves (the live record and the retained record with name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID and ATTACH_SYNTAX); the entries of both storages before and after O10 and before and after C3; `switched_utc`, `retain_until_utc` and the C5 command; the pipelines not recreated because no credentials were given; the definer per view (Views tab) and per routine (Routines tab); the workspace answers O1 recorded. A value the journal lacks reads "not recorded"; C2 before C3 leaves the C3 listing rows that way, and `s2rebuild report` writes both files again later.
- Retention: C2 records `retain_until_utc`, the switch time plus `closeout.retain_days`. The kit route records the switch time at S3. On the hand route (the printed block or the notebook), `gate O10 --pass` records the time of the gate, which is never before the switch.
- Gate: reports written.
- Re-enter: `s2rebuild step C2` writes the reports again from the same journal.
- When it fails: fix the cause and rerun. To record C2 by hand: `s2rebuild gate C2 --pass --note TEXT --override NOTE`. Gate refuses C2 without `--override`; the override is journaled.

C3 End state

- Identity: owner.
- Purpose: prove that the run left exactly two entries on the workspace, the live database and the retained copy, and record the listing.
- When: after C2. No waiting period: the retention period applies to C5, not to C3.
- Gate: the live name resolves to the recorded live record (a pitr record of the replacement) and holds no other record; the retained name holds the recorded retained record READONLY (a resume record of the original storage) and no other record; the entries with IS_DROPPED = 0 on the original storage and on the replacement storage are exactly those two; a write on the retained name is refused (the O3 write probe, run as the runner). The listing before and after goes into the journal as `endstate_listing` and into the final report.
- Statements: none (reads and the write probe). C3 never detaches, attaches or drops a database.
- Re-enter: `s2rebuild step C3`. The gate is reads only, so a rerun is free.
- When it fails: the note names the check, the expected value and the observed value, and the listing of both storages; find out who attached or moved the extra entry. The retained copy attached READWRITE, or a record that differs from the recorded one, means someone moved a database; C3 never detaches, attaches or drops a database the run did not record.
- Then: keep the evidence folder. The retained copy stays until the customer deletes it (C5).

C4 Remove the migration account

- Identity: owner.
- Preconditions: C3 passed; no view and no routine in the live database has the runner as its definer.
- Statements: a check that the account still holds privileges, the definer check, DROP USER, a check that zero privileges remain.
- When it stops at the definer check: a view or a routine whose definer is gone can no longer be executed. List them with `SELECT TABLE_NAME, DEFINER FROM information_schema.VIEWS WHERE TABLE_SCHEMA = '<live database>'` and `SELECT ROUTINE_NAME, DEFINER FROM information_schema.ROUTINES WHERE ROUTINE_SCHEMA = '<live database>'`, recreate each one the runner defines as a durable account (the admin or the application account), then rerun `s2rebuild step C4`. The kit never creates a view or a routine as the runner, so the check finds one only after a hand run.
- Gate: C3 passed before; DROP USER done and zero privileges remain. `s2rebuild step C4` records it, and so does `s2rebuild account-sql --drop --execute` (the privileges before and after in the gate note).
- Then: delete the Keychain item that held the runner password.

C5 Delete the retained original (optional, the customer)

- Command: `s2rebuild cleanup` lists what the run left after C3: the live record, the retained copy with its record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), its recovery syntax, its size in GiB from the capture, the switch time, `retain_until_utc` and the C3 status. `s2rebuild cleanup --delete <retained>` deletes the retained copy.
- Identity: owner. The kit never runs it by itself; no step and no configuration key deletes the retained copy.

- Refused, with nothing issued and the reason journaled: a name other than the run's retained name; no retained record in the journal; no `retain_until_utc` (C2 records it); today before `retain_until_utc` ; C3 not passed; a record under the name that differs from the recorded one, or not READONLY.
- Two confirmations: the `--delete NAME` flag, then the name typed again at the prompt. Any other input keeps the copy, records "kept by customer" and exits 0. `--yes` does not apply; without a terminal the command refuses.
- Statements, each behind its record check (a READ ONLY attachment cannot be dropped, Helios fact 28): DETACH DATABASE retained; the detached record's own ATTACH_SYNTAX (read write); a check that the record is attached READWRITE; DROP DATABASE retained; checks that the record is dropped and the name free, and that the live name still resolves to the live record. The journal records `retained_deleted`.
- Then: the workspace holds the live database alone. The dropped record stays recoverable with `ATTACH DROPPED DATABASE` for the retention period of the engine.

Abandon before the switch

- Purpose: back out a run that stops before O10, for example to start again on a larger workspace.
- Identity: owner.
- Before: `s2rebuild pause`, then wait until the copy step ends. abandon takes the copy lock and refuses while a copy runs.
- Statements: checks that the source name holds the recorded record, that the replacement name holds the record O5 recorded or nothing, and that no branch was made from the replacement (a branch means the switch attached the live name from it); DETACH and ATTACH of the source READWRITE on its recorded STORAGE_ID, each behind a record check (only the ATTACH after a stop between them; checks only when the source is already READWRITE); DROP DATABASE of the replacement, skipped unless its name holds the recorded record; checks that the replacement record is dropped and the source is READWRITE as its recorded record.
- Commands: `s2rebuild abandon` prints the block; `s2rebuild abandon --execute --note TEXT` runs it. Gate: every check ok, and the log line "run ... abandoned".
- Then: `s2rebuild account-sql --drop --execute` removes the runner. `run`, `step` and `gate` refuse the abandoned run; a new run starts at P1.
- Refuses: once the switch may have begun (use `s2rebuild owner-sql O10 --rollback`), and before P2 recorded the source STORAGE_ID.
- Re-enter: rerun the same command; it reads the attachment state and finishes what is left.